IT305 LAB : Socket Programming Project

Fault Tolerant Topic-based File Distribution Service

This project will be carried out in the group of 5-6 students. We already have groups of three students each in the lab. The project group will be formed by merging the two groups.

The project emphasizes TCP socket programming, multi-threaded servers, and performance measurement. The project has two interlinked parts as mentioned below. Part-1 focuses on basic file distribution services, whereas Part-II adds fault tolerance and performance enhancements.

Part – 1 Basic Topic Based File Distribution

In the part -1, aim is to implement a topic-based file distribution system where the server maintains a set of topics, each mapped to a directory containing multiple files on the topic. Clients request a topic (see dataset below), and the server responds by sending all files belonging to that topic. The server is to be implemented in both single-threaded and multi-threaded modes to study their effect on throughput.

It is expected that you already know multi-threaded application design which is required for implementing multi-threaded servers. If required, please review the POSIX pthread library specification (https://pubs.opengroup.org/onlinepubs/9799919799/) and refer to tutorials such as LLNL POSIX Threads Guide for programming examples (https://hpc-tutorials.llnl.gov/posix/).

DATASET:

- We will use the dataset available at https://www.kaggle.com/datasets/alessiocorrado99/animals10 (also posted on Google classroom). The dataset has 10 directories, each containing a set of images of a specific type of animal. Each animal type represents a topic.

PART I: Single and Multi-threaded Topic-based File Server

- Implement a TCP socket-based server that maintains a list of topics (keys). In our case, the animal types in the dataset represent topics. For every topic (key), the associated value is the absolute path of the directory containing its files.

Key (Topic)	Value (directory path)
Cane	/home/manish/lab-6/raw-img/cane
Cavallo	/home/manish/lab-6/raw-img/cavallo
Elefante	/home/manish/lab-6/raw-img/elefante
Farfalla	/home/manish/lab-6/raw-img/Farfalla
Gallina	/home/manish/lab-6/raw-img/Gallina
Gatto	/home/manish/lab-6/raw-img/Gatto
Mucca	/home/manish/lab-6/raw-img/Mucca
Pecora	/home/manish/lab-6/raw-img/Pecora
Ragno	/home/manish/lab-6/raw-img/Ragno
Scoiattolo	/home/manish/lab-6/raw-img/Scoiattolo

- When a client requests a topic:

    - The server looks up the corresponding directory path.
    - It sends all files in that directory to the client.
- Implement the Server program using single thread to serve requests from all the clients sequentially.
- Further, implement the multi-threaded Server program to serve requests from multiple clients simultaneously. You may create separate thread to handle each client and/or thematically different tasks separately. You will explain your design decision during analysis.
- Implement the client program that:
    - Connects to the server.
    - Issues a request in the form: GET <topic>.
    - Receives all files belonging to the requested topic (creates the directory corresponding to the topic and populates it with the received file).
- Run experiments on a group of three computers:
    - One of the node will execute server (single/multi threaded).
    - Multiple clients will run on each of the other two nodes.
    - Single-threaded server serves clients sequentially.
    - Multi-threaded server serves multiple clients concurrently.
- Measure and compare performance of single/multi threaded server implementation:
    - Measure time required to send data to requesting client. You may use clock_gettime() function available from <time.h> or similar library function.
    - Count the number of data bytes sent on the socket during this time.
    - Use the above two to compute aggregate file transfer rate
    - Prepare a table or graph to record aggregate file transfer rate v/s number of clients for the single threaded server and multi-threaded server.
    - Analyze and comment about the performance of these server implementations.

PART II: Fault-Tolerant File Transfer (Handling Disconnection and Resume over the Multi-threaded Server)

Building on the multi-threaded server from Part I, this part requires the server and client to remain correct and make forward progress even when a disconnection occurs at any point in time during the transfer.

Fault injection: the server shall probabilistically terminate/interrupt an active client connection during file transfer. The probability of failure shall be controlled by the <failure probability> command-line argument provided to the server. The implementation should document when and how the failure decision is made.

All cases below must run on the multi-threaded server, with one thread serving each client.

- Case 1: No Session Management. No metadata about an in-progress transfer is retained by either side. After a failure, the transfer restarts from the beginning.
- Case 2: With Session Management. Client and server maintain some state so that after a failure, the transfer resumes from the previous checkpoint instead of restarting from the beginning.
    - Session management means the client and server keep track of an in-progress transfer instead of forgetting everything the moment a connection drops.
    - A checkpoint is a small saved record of progress - typically the topic, the current file name, and the byte offset already sent/received for that file.
    - This checkpoint state is updated periodically as the transfer proceeds (e.g., after every file, or after every few KB), on the server, the client, or both.

    - When the client reconnects after a failure, it reports its last known checkpoint; the server validates it and resumes sending from that byte offset instead of from the first byte of the topic.
    - No data that was already successfully transferred and recorded needs to be sent again.
- Measure and compare performance of your implementation with and without session management by varying the fault injection rate. Compare the total bytes transmitted, useful bytes successfully delivered, redundant bytes retransmitted, completion time, throughput v/s overhead, and compare your results with Part 1 values.

To improve performance further, you may consider the following **performance enhancements** in the Part-II – Case 2. We call it as **Part-II – Case 2 – Enhanced**

- Socket mode: you may use either blocking or non-blocking sockets. In the Blocking sockets, the socket API calls (accept/send/recv) pause the calling thread until the operation completes, so the thread does nothing else while it waits. In the Non-blocking sockets, the socket APIs returns immediately even if it cannot complete right away (e.g., no data yet).
- Parallel TCP Connections: Instead of a single connection, the transfer for one topic uses multiple simultaneous TCP connections between client and server. The design and implementation approach (how many connections are used, how work is split across connections, how data is reassembled, how a failed connection is recovered) is left to you - document and justify whatever choices you make.
- Report the performance enhancements used. Measure and report the performance improvement. Support your claims with appropriate justification and result analysis.

Submission:

There are two parts of the project submission:

- Design document
    - Deadline: 27 September 2026
- Final code and result analysis
    - Deadline: 16 October 2026

Your design document must cover:

- Server data structures used to map topics to directories.
- Client-server message formats, for Part I and for every case of Part II (Case I, Case II, and Case II – Enhanced).
- Fault injection mechanism planned to be used in your work.
- Details of session management protocol: sequence and types of communicated messages, message formats used by the client and server, what information is stored by the server/clients.
- Performance enhancements planned in the implementations
- Any extra or additional properties/features of your implementation beyond the minimum requirements (for example: logging, configurability, graceful shutdown, extensibility to new topics) that you plan to include, and why.
- How your code will look: the overall module/file structure, the key functions or classes you plan to write, and how the single-threaded, multi-threaded, and fault-tolerant logic fit together.
- A short write-up of your design thought process for the project - the alternatives you considered for each case and why you chose your final approach.

The design document (.pdf file) must be named **Design_Document_Group_XXX_YYY_.pdf**, where XX and YY are the group numbers of the two lab groups that have merged to form the

project group. For example group 5 and group 102 have decided to form the project group, theur design document will be named **Design_Document_Group_005_102_.pdf.**

Submission of Final Code and Result Analysis:

A final working code is expected as a part of this submission. The submissions with errors (e.g., compilation errors/warnings, run time errors, etc.) will be altogether discarded.

Read this section carefully and follow the naming convention exactly - submissions that do not follow it may be rejected by the auto-grader or delayed in evaluation.

- Submission archive: FinalSubmission_Group_<XXX>_<YYY>.<zip or tar.gz>. For example, FinalSubmission_Group_005_102.tar.gz.
- The submission archive should include everything as mentioned below. Do not submit loose/individual files.
  - Report file: FinalReport_Group_<XXX>_<YYY>.pdf - one consolidated PDF report summarizing implementation details of Part I and Part II (all cases), along with the performance comparison and analysis of each implementation scenario.
  - Source code folders: Part1/, Part2_Case1/, Part2_Case2/, Part2_Case2_Enhanced/, one folder per case, each with its own server code, client code, and build script (Makefile)
  - Results folder: Results/. This folder should contain raw performance logs (.csv) and graphs (.png/.pdf) for each case. Example file names are Partname_CaseName_throughput.png and Partname_CaseName_overhead.png.
  - README file: README.md file containing build/run instructions for every case and the fault-injection mechanism used for testing.

Folder structure of your final submission will be as follows:

- FinalSubmission_Group_<XXX>_<YYY>/
  FinalReport_Group_<XXX>_<YYY>.pdf
  README.md
  Part1/
  Part2_Case1/
  Part2_Case2/
  Part2_Case2_Enhanced/
  Results/

Standard CLI Interface (Required So Your Submission Can Be Auto-Tested)
The grading script builds and runs your programs against a reference dataset it supplies, so nothing in your server or client may hard code a file path, port number, or IP address – every one of these must come from a command-line argument, exactly as follows. Deviating from this interface (extra required arguments, interactive prompts, hard coded paths/ports, differently named executables) will make the automated check fail for that case, regardless of whether your code works when you run it manually.

- Required invocation of server program
  - for Part 1: *./server <port> <topics_root_dir>*
  - for Part 2 all cases: *./server <port> <topics_root_dir> <failure probability>*
  - Here, the *topics_root_dir* contains one sub-folder per topic; the sub-folder name is the topic key.
  - The *failure probability* specifies how frequently the client-server session is interrupted.
- Required invocation of client program

    - *./client <server_ip> <server_port> <topic> <output_dir>*
    - The invocation fetches *topic* and writes files under *output_dir/topic/*
- Running make inside each Part1/, Part2_Case1/, Part2_Case2/, Part2_Case2_Enhanced/, folder must produce (or update) exactly two executable files in that same folder, named "server" and "client". A wrapper shell script invoking a compiled binary or an interpreter is fine, as long as it is named exactly server or client and is directly runnable as per the above invocations.
- Extra optional trailing arguments are allowed for case-specific configuration (e.g., number of parallel connections for Case 2 Enhanced), as long as sensible defaults exist so the two invocations above, used as-is, always work.

Final Report Content Checklist:

- The report must clearly separate Part I and Part II, and within Part II must clearly separate Case 1, Case 2, and Case 2- Enhanced
- The report should have a Cover page with information about Group numbers, full name and roll number of every group member, and a one-line note on their individual contribution.
- Design section: This should be based on your design document submitted earlier, and should contain implementation level detail. In case there are deviations from your earlier design submission, mention those here.
- Performance Analysis Section: the throughput and overhead tables/graphs for Part I and for all the cases of Part II with detailed analysis and summary of observations.

Evaluation:

- Total marks: 20
- Design document will be evaluated against a model solution, and marks will be awarded out of 8. Model design will be shared with students to guide the implementation.
- The final submission will be evaluated through automated scripts and rigorous viva about the implementation. The marks will be warded out of 12.

IMPORTANT ABOUT ACADEMIC INTEGRITY

- **All code and report content must be the group's own work. Reusing code from another group, a previous offering of this course, or an online source without attribution will be treated as plagiarism under institute policy. Every member is jointly responsible for the entire submission, not only for the case(s) they personally implemented.**